

Répondre de façon concise mais précise aux questions suivantes en utilisant les cadres. Utiliser les notations du cours. Aucun document n'est autorisé, hormis un aide-mémoire manuscrit de format A4 recto seul constitué par chaque étudiant. Ce sujet comporte 2 pages. Commencer par remplir le bloc d'identification.

1. Répondez aux questions suivantes en justifiant brièvement la réponse. Bonus pour trois bonnes réponses.

- **Donnez un exemple $\mathcal{L}$ d'un langage autre que $\{a^n b^n \mid n > 0\}$ qui est algébrique mais pas rationnel.**

Lors de la séquence sur le Lemme de pompage on avait vu un tel langage : le langage des palindromes. Plus précisément, on avait vu lors de cette séquence qu'il n'était pas rationnel, puis on avait vu lors de la séquence sur les grammaires algébriques qu'il était algébrique.

On peut mentionner aussi le langage de Dyck qui généralise $\{a^n b^n \mid n > 0\}$. Lui, on en n'a pas parlé dans la séquence sur les rationnels, mais dans la séquence sur les algébriques (au tout début).

- **Proposez une grammaire algébrique qui engendre ce langage.**

Palindrome $\rightarrow 's_i'$ Palindrome $'s_i'$	% pour chaque $s_i \in \Sigma$
Palindrome $\rightarrow 's_i'$	% pour chaque $s_i \in \Sigma$
Palindrome $\rightarrow \varepsilon$	

Pour le langage de Dyck voir le diaporama sur les grammaires.

- **Donnez un exemple de langage $\mathcal{L}' \supset \mathcal{L}$ qui est rationnel.**

Σ^* est un tel exemple, le plus simple à concevoir sans doute.

L'important est de comprendre que les inclusions entre langages n'ont rien à voir avec les inclusions entre familles de langages. On a Rationnels $\subset$ Algébriques (strictement puisque qu'on connaît des langages algébriques qui ne sont pas rationnels), mais ça n'empêche pas un langage rationnel de contenir un langage algébrique qui n'est pas rationnel (strictement encore, car un langage ne peut pas à la fois être et ne pas être rationnel). Ici, il faut bien dire « langage algébrique qui n'est pas rationnel » car puisque tout langage rationnel est algébrique tout langage rationnel contient (non strictement) un langage algébrique.

2. Quelle est le principal ajout conceptuel des grammaires algébriques étendues par rapport aux grammaires algébriques de base ?

Le principal ajout conceptuel des grammaires algébriques étendues par rapport aux grammaires algébriques de base est que les corps de règles prennent la forme d'expressions régulières plutôt que la forme d'une séquence de symboles terminaux et non terminaux.

3. Listez deux avantages des grammaires algébriques étendues par rapport aux grammaires algébriques de base. Bonus pour deux bonnes réponses.

Avantage 1 : D'une manière générale, les grammaires algébriques étendues permettent une notation plutôt plus compacte qu'avec les grammaires algébriques de base. Par exemple, elles dispensent de créer des non-terminaux spécifiques rien que pour dire « liste de *ceci* » ou « liste de *cela* ». Il suffit d'écrire *ceci** ou *cela**. Cela peut évidemment être plus compliqué si il y a des séparateurs ou des terminateurs, mais c'est l'idée. Par exemple, une déclaration à la Java pourra être décrite comme :

$$(Decl', ')* Decl';$$

Avantage 2 : On a vu en fin de la séquence sur l'analyse syntaxique LL(1) qu'elles permettraient plus facilement de respecter les critères LL(1).

4. Rappelez les deux règles de dérivation de la stratégie LL(1).

Règle 1 : Si le symbole en sommet de pile est un symbole terminal, on a :

$$(Pile \bullet a, a \bullet m) \rightarrow^a (Pile, m), \text{ le symbole } a \text{ est consommé.}$$

Règle 2 : Sinon, le symbole en sommet de pile est un symbole non-terminal, on a :

$$(Pile \bullet A, m) \rightarrow^\varepsilon (Pile \bullet \text{miroir}(\gamma_i), m), \text{ si } A \rightarrow \gamma_i \text{ est une règle de la grammaire.}$$

Ces deux règles ont des conditions d'application disjointes puisque le symbole en sommet de pile est forcément terminal ou pas, mais pas les deux à la fois. Donc, le contenu de la pile dit exactement quelle règle choisir.

La première règle ne pose pas réellement de difficulté, et elle ressemble fort à l'unique règle de dérivation des automates finis.

Mais la seconde règle est embarrassante si il existe plusieurs règles pour le symbole A car rien ne dit laquelle choisir. D'où la problématique de l'analyse LL(x) ; trouver dans les x premières lettres du mot qui reste à analyser un indice pour choisir la bonne règle de grammaire. Cette règle est aussi embarrassante si γ_i commence par A car alors le résultat est de remettre un A en sommet de pile, et donc de créer une boucle infinie.

5. Qu'est-ce que la forme normale de Greibach ?

C'est une forme de grammaires plus restreintes que les grammaires algébriques de base, mais vers laquelle toute grammaire algébrique peut être convertie. Cette forme est la suivante :

$$\Sigma_{NT} \rightarrow \Sigma_T \Sigma_{NT}^* \text{ ou } \Sigma_{NT} \rightarrow \varepsilon$$

Autrement dit tous les corps de règles d'une grammaire en forme normale de Greibach commencent par une lettre terminale ou sont vides.

6. Qu'est-ce que la récursivité à gauche ? Pourquoi est-ce grave dans le cadre LL(1) ? Comment la forme normale de Greibach peut-elle aider à résoudre cela ?

Une grammaire est récursive à gauche si elle comporte des règles dont le corps commence par le même symbole que celui de la tête. C'est grave, car ça fait boucler la seconde règle de dérivation LL(1) (voir question 4). La forme de Greibach apporte une solution radicale puisqu'elle interdit d'avoir des symboles non-terminaux en début de corps de règle !

Plus généralement, on considère aussi les récursivités gauches indirectes, comme dans

$$\begin{aligned} A &\rightarrow B \gamma_1 \\ B &\rightarrow A \gamma_2 \end{aligned}$$

Elles ont le même inconvénient que les récursivités gauches simples, mais je n'en ai pas parlé en cours. En effet, partant d'une pile $Pile \bullet A$, on arrive à une pile

$$Pile \bullet \text{miroir}(\gamma_1) \bullet \text{miroir}(\gamma_2) \bullet A,$$

et on reboucle.

Soit l'alphabet $\Sigma_{id} = \{ X, 0, \dots, 9 \}$. On appellera collectivement les 0, ..., 9 des 'digit'.

Soit $\mathcal{L}_{id}$ le langage des identificateurs de variables sur Σ_{id} . Ex. X, X0, X1, X125.

$\Sigma_{bool} = \{ \text{AND}, \text{OR}, 0, 1, (,), =, ; \}$.

Soit $\mathcal{L}_{expr}$ le langage des expressions booléennes sur Σ_{bool} et $\mathcal{L}_{id}$. Ex. X, X1 AND X2, X1 AND X2 OR X3, X1 AND (X2 OR X3).

Soit $\mathcal{L}_{prog}$ le langage des programmes booléens sur Σ_{bool} et $\mathcal{L}_{expr}$. Ex.

$$X = 1 ; X12 = X1 \text{ AND } X2 ; X73 = X12 \text{ OR } X3$$

On souhaite composer une grammaire algébrique LL(1) qui décrit le langage $\mathcal{L}_{prog}$.

7. La grammaire suivante sensée être LL(1) et décrire le langage $\mathcal{L}_{prog}$ présente un certain nombre d'anomalies. Listez-les.

- | | |
|--|---|
| 1. $\text{Id} \rightarrow 'A' \text{ Num}$ | 4. $\text{Expr} \rightarrow \text{Expr OP Expr}$ |
| 2. $\text{Num} \rightarrow \text{digit Num}$ | 5. $\text{Expr} \rightarrow '0' \mid '1' \mid \text{Id} \mid '(' \text{Expr} ')'$ |
| 3. $\text{OP} \rightarrow \text{'AND'} \mid \text{'OR'}$ | 6. $\text{Instr} \rightarrow \text{Id '=' Expr}$ |

7. $\text{Prog} \rightarrow \text{Prog} ';' \text{Instr}$

8. $\text{Prog} \rightarrow \text{Instr}$

Règle 1 : selon Σ_{id} le 'A' devrait être un 'X'. Ici, la grammaire est fausse au sens où elle n'engendre pas le langage désiré.

Règle 2 : on voit bien que Num devrait engendrer une suite de chiffres, mais il manque un cas d'arrêt. Ici, la grammaire est fausse au même sens.

Règle 4 : on reconnaît une forme familière vue en cours et en TD, celle d'une règle très ambiguë. Ici, la grammaire engendre le bon langage, mais de façon ambiguë.

Règles 4-5 et 7-8 : elles ne sont pas LL(1). Par exemple, les règles 7 et 8 peuvent toutes les deux engendrer un mot qui commence par un 'X'. Et ici, la grammaire engendre le bon langage, sans ambiguïté, mais elle n'est pas compatible avec la stratégie d'analyse LL(1).

8. Expliquez brièvement comment vous pourriez répondre à chaque anomalie.

Règle 1 : remplacer le 'A' par 'X'.

Règle 2 : ajouter un cas d'arrêt. Là, il faut se demander si la suite de chiffres peut être vide ou non, car ça ne sera pas le même cas d'arrêt. Réfléchissez 2 minutes avant de lire la réponse à la question 9.

Règle 4 : premier réflexe, former un ensemble de règles qui rend compte de la priorité des opérateurs. Généralement, AND est considéré prioritaire sur OR. Mais cette démarche produit une grammaire qui n'est pas LL(1) (vu en TD).

Règles 4-5 et 7-8 : Il faut aller plus loin et utiliser une solution radicale : mettre ces règles en forme de Greibach, ou utiliser la forme étendue des grammaires algébriques.

9. Proposez une variante de cette grammaire qui ne présenterait plus d'anomalies. Ne montrer que les différences avec la grammaire originelle.

$\text{Id} \rightarrow \text{'X' digit}^*$	% les exemples montrent % que la suite de digits peut être vide
$\text{OP} \rightarrow \text{'AND' 'OR'}$	
$\text{Expr} \rightarrow \text{ExprAnd ('OR' ExprAnd)}^*$	% des ExprAnd séparés par des OR
$\text{ExprAnd} \rightarrow \text{ExprAtom ('AND' ExprAtom)}^*$	% des ExprAtom séparés par des AND
$\text{ExprAtom} \rightarrow \text{'0' '1' Id '(' Expr ')'}$	
$\text{Instr} \rightarrow \text{Id '=' Expr}$	
$\text{Prog} \rightarrow \text{Instr (';' Instr)}^*$	% encore des trucs séparés par des machins

En **rouge**, ce qui change, en **gris léger**, ce qui ne change pas et que vous pouviez ne pas montrer.

Répondre de façon concise mais précise aux questions suivantes en utilisant les cadres. Utiliser les notations du TP. Aucun document n'est autorisé, hormis un aide-mémoire manuscrit de format A4 recto seul constitué par chaque étudiant. Ce sujet comporte 2 pages. Commencer par remplir le bloc d'identification.

1. Répondez aux questions suivantes relatives au package `regex` de Java en justifiant brièvement chaque réponse. Bonus pour 3 bonnes réponses.

- **À quoi sert la méthode `compile` ?**

Si `re` est une chaîne qui représente une expression régulière,

`Pattern.compile(re)`

va reproduire une représentation interne de `re` pour un usage ultérieur.

Vraisemblablement, cette représentation interne sera un automate fini. Fabriquer cette représentation a un coût, et on n'utilise la méthode `compile` que si on a l'intention d'utiliser plusieurs fois la même expression régulière.

- **À quoi sert la méthode `matcher` ?**

Si `p` est un `Pattern`, compilé ou non, et `t` un texte (une chaîne de caractères), `p.matcher(t)` va produire le résultat de la comparaison de l'expression régulière à l'origine de `p` et de `t`.

- **À quoi servent les méthodes `start` et `end` ?**

Les méthodes `start` et `end` retournent la localisation d'une occurrence d'un motif recherché dans une chaîne de caractères. La méthode `start` retourne l'indice du premier caractère de l'occurrence du motif, et `end` retourne l'indice du premier caractère qui suit l'occurrence du motif.

2. Quel effet a le quantificateur $\{n, m\}$?

Ce quantificateur spécifie que le motif qui précède peut être répété entre n et m fois. Par défaut, n vaut 0, et m l'infini. Ainsi `r{, 12}` signifie au plus 12 fois `r`, et `r{12, }` signifie au moins 12 fois `r`.

3. Rappelez le rôle de `^` et `$` dans les expressions régulières.

Dans une expression régulière, `^` et `$` signifient respectivement le début et la fin de la chaîne dans laquelle on recherche un motif. Ainsi `^r` spécifie que `r` doit être trouvé en début de chaîne, et `r$` spécifie que `r` doit être trouvé en fin de chaîne.

Ne pas confondre avec un autre usage de `^` : celui de signifier le complément ensembliste dans une notation d'ensemble de caractère. Ex. `[A-Z&&[^QUI]]` signifie toutes les majuscules sauf Q, U et I.

4. Proposez une expression régulière pour les numéros de téléphone français avec ou sans indicatif international. Ex. 01 23 45 67 89 ou +33 1 23 45 67 89.

$$\mathcal{RE}_{\text{NumBase}} = "\d (\d{2})^{3}\d{2}"$$
$$\mathcal{RE}_{\text{NumLocal}} = "0" + \mathcal{RE}_{\text{NumBase}}$$
$$\mathcal{RE}_{\text{NumInternational}} = "+33 " + \mathcal{RE}_{\text{NumBase}}$$
$$\mathcal{RE}_{\text{NumTel}} = \mathcal{RE}_{\text{NumLocal}} \mid \mathcal{RE}_{\text{NumInternational}}$$

5. À quoi sert la méthode `find` ?

La méthode `find` est une méthode d'un matcher obtenu grâce à la méthode `matcher`. (voir la question 1). Elle sert à rechercher itérativement dans un texte des occurrences d'un motif. La méthode rend vrai ou faux selon qu'elle a trouvé une occurrence du motif ou non. Le matcher enregistre les caractéristiques de la dernière occurrence trouvée. On peut les consulter à l'aide des méthodes `start` et `end` de la question 1.

6. Rappelez schématiquement ou par un exemple l'usage de `find` dans une boucle `while`.

La méthode `find` va retourner la condition de sortie de la boucle `while` :

```
former un matcher m
while (m.find() trouve une nouvelle occurrence) {
    faire quelque chose avec la description de l'occurrence
    par exemple, en utilisant m.start() et m.end()
}
```

7. Soit un texte `source`, une regex `cible`, présentez les grandes lignes d'un code qui liste toutes les occurrences de `cible` dans `source`.

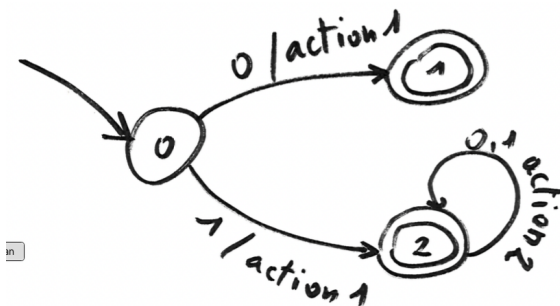
```
Pattern p = Pattern.compile(cible) ;
Matcher m = p.matcher(source) ;
while (r = m.find() ) {
    publish(r.start(), r.end(), source)
}
```

C'est essentiellement la première question du TP `regex`. La fonction `publish` reste à définir pour former la liste.

8. Quelle est la principale différence conceptuelle entre les automates du TP Programmation par automate et les automates théoriques du cours ?

Les transitions des automates du TP sont décorées d'un numéro d'action. Dans la boucle générale de l'algorithme d'acceptation ces actions sont exécutées au fur et à mesure des étapes de dérivation (voir le sujet de TP Programmation par automate ou la fin du diaporama sur les automates).

9. Proposez un automate qui accepte les séquences de 0 et de 1 qui ne commencent pas par 0, sauf si la séquence est réduite à 0. Donner la représentation graphique de l'automate, puis sa construction dans le cadre du TP Programmation par automate.



```
put_final(1)
put_final(2)
unArc(0, '0', 1, 1)
unArc(0, '1', 2, 1)
unArc(2, '0', 2, 2)
unArc(2, '1', 2, 2)
```

C'est comme dans le TP en plus simple puisqu'il n'y a que deux chiffres et rien d'autre. Pour la partie strictement automate (c-à-d. automate sans action) c'est exactement comme un exemple du début du diaporama sur les automates.

10. Précisez des actions pour l'automate de la question 9 de telle sorte que l'acceptation d'une séquence de bits calcule en même temps la valeur du nombre lu.

Action 1 : $\text{valeur} = \text{tete} - '0'$

Action 2 : $\text{valeur} = 2 * \text{valeur} + (\text{tete} - '0')$

Action 1 initialise la variable valeur, et Action 2 la complète. Une autre solution consiste à redéfinir la fonction `actionInitiale()` pour initialiser valeur à 0, et à utiliser Action 2 pour toutes les transitions.